Phase 3 | Technologies, Foundations & Design Decisions

Learn the tools and the reasoning behind the implementation.

FastAPI Edition | Repository-grounded study material | Snapshot: 2026-10-24

1. FastAPI, ASGI, and Uvicorn

FastAPI is an HTTP framework built on ASGI-compatible Starlette. ASGI defines an asynchronous callable interface between server and application, allowing requests, responses, lifespan events, and long-lived connections. Uvicorn is the ASGI server that accepts network connections and invokes the app. `uvicorn run:app` means import module `run`, then use its `app` object.

Why it fits: concise routes, Starlette request/response/static infrastructure, generated OpenAPI docs, and easy integration with async I/O. Trade-off: declaring a handler `async` does not make synchronous CPU work nonblocking. MetroInsights explicitly offloads its synchronous inference helper through `run_in_threadpool`; truly CPU-heavy work may still need separate workers or processes.

Flask comparison: Flask's WSGI request model is mature and straightforward. FastAPI is a sensible fit for a JSON-first API and typed contracts, but this code currently uses raw `Request` rather than Pydantic request bodies, so it is not yet benefiting from full request schema validation.

2. Request lifecycle and concurrency

A request passes from browser -> Uvicorn -> ASGI app/router -> handler -> response. An async function yields control only when awaiting cooperative async operations. Blocking Pandas, sklearn, or file work inside the event loop would stall other requests. The project moves core synchronous prediction work to a threadpool. Threads help keep the loop responsive; Python CPU-bound work may remain limited by the GIL, and large arrays/models can increase memory pressure. Measure before adding a task queue.

3. Jinja2, HTML, JavaScript

Jinja2 renders templates with server-provided context. StaticFiles serves CSS/JS. Server rendering is simple for initial dashboard state; JavaScript fetches JSON for dynamic updates. Keep the API contract explicit: frontend expects named JSON fields, units, and state values. A backend field rename can silently break charts. Add API contract tests and browser smoke checks as an improvement.

4. Pandas and NumPy

Pandas provides labeled tabular transformations, joins, grouping, datetime parsing, and CSV loading. NumPy provides vectorized numerical operations, trigonometric encodings, arrays, and metric support. Training uses both for millions of synthetic rows; inference also constructs a one-row DataFrame to honor sklearn's feature names. Trade-offs: memory use, dtype coercion, implicit alignment, and overhead on tiny one-row operations. Keep expensive preparation offline where possible.

5. Scikit-learn and the regressor

`HistGradientBoostingRegressor` is a tree boosting estimator that grows decision trees sequentially, each stage reducing residual error from previous stages. Histogram binning reduces candidate split cost and supports missing values. It can model nonlinear interactions such as hour x station type without manual linear interaction terms. Risks include overfitting to generated rules, extrapolation weakness beyond represented categories/ranges, sensitivity to data-generation patterns, and less natural interpretability than a simple linear model.

Ridge is a linear model with L2 penalty: it can provide a stable baseline but cannot naturally express arbitrary thresholds/interactions unless features encode them. Historical station/day/hour mean is a strong domain baseline. Compare models on the exact same chronological split and inspect errors by station, hour, calendar type, and demand level, not only global averages.

6. Feature engineering foundations

- **Cyclic encoding:** hour 23 and hour 0 are adjacent, but integer distance says 23. Use sine/cosine: `sin(2πh/24)`, `cos(2πh/24)` so the cycle wraps smoothly. Same idea for day-of-week and month.
- **Calendar flags:** weekend, workday, holiday, holiday class encode known schedule context; quality depends on calendar coverage and correctness.
- **Station/line indices:** convert category labels to numeric IDs. For a tree model, integer indices can suggest artificial order; one-hot or native categorical encoding could be compared.
- **Historical aggregates:** station mean, station-hour-weekday mean, station-month mean summarize prior demand. They must be computed only from information available at forecast time.
- **Typology/interchange:** compact station descriptors add context but reflect project-assigned classifications.

7. Metrics and chronological validation

MAE is mean absolute error, interpretable in target units. RMSE squares errors before averaging and penalizes large misses more. R^2 measures explained variance relative to predicting the test mean; it can look high when the target is smooth. MAPE divides by actual values and becomes unstable/undefined near zero; the pipeline computes it only on positive labels. Chronological holdout better approximates predicting later dates than random splitting, but synthetic distribution shift can differ from real future conditions.

8. Joblib and artifact compatibility

Joblib serializes Python objects efficiently, particularly NumPy-heavy models. It is suitable for trusted, version-controlled artifacts; loading an untrusted pickle/joblib file can execute code. The model, metadata feature ordering, category mappings, sklearn version, and code must be compatible. Pin dependencies and store artifact version/hash/training provenance. A missing artifact currently can break module import.

9. Graph representation and mapping

An adjacency list maps each graph node to neighbor-weight pairs. Here node identity is `(canonical station, line)`. This is a state expansion: a physical station on two lines is two states plus transfer edges. Mapping aliases are a data-integration problem, not merely cosmetic text cleanup: a failed alias deletes edges and changes reachability. Track normalization collisions and unmatched names explicitly.

Haversine computes great-circle distance between latitude/longitude points. The graph uses it as a proxy for adjacent-station distance and divides by a fixed average speed to produce time. This ignores track curvature, dwell, signaling, actual timetable, and station-specific transfer walks. Bad coordinate detection and a fallback reduce extreme estimates but add assumptions.

10. Dijkstra foundations

Dijkstra is correct when edge weights are non-negative. It uses a priority queue, not repeated full sorting. Relaxation checks whether `dist[u] + w` improves a neighbor's known distance. A visited set settles each node once under the standard non-negative-weight condition. Multiple source nodes all start at zero. With a binary heap and adjacency lists complexity is `O((V+E)log V)`. If weights became negative, use another algorithm; if only one destination were needed, bidirectional search might help, though this network is small.

11. Why CSV/JSON rather than a database?

For a local prototype and static reference tables, CSV/JSON are transparent, versionable, and easy to inspect. They do not provide concurrent transactions, indexing, relational integrity, or safe multi-user writes. Since current workflows are read-mostly and no user accounts exist, a database is not mandatory. If adding live observations, user configurations, or concurrent writes, move authoritative data to PostgreSQL and use migrations/constraints.

12. Interview prompts

Explain ASGI versus WSGI; async versus threadpool; why Dijkstra fits; why a station-line state; why chronological split; what MAE/RMSE/R^2 mean; why synthetic validation is limited; how category IDs affect trees; how joblib risks differ from JSON; when a database becomes useful.

How to use this series

Read Phase 1 first, then Phase 2 while opening the source files. Use Phases 3, 5, 6, and 7 to deepen technical understanding; rehearse with Phases 4 and 8; revise with Phase 9. The editable Markdown beside each PDF is the source of truth for corrections.

Evidence and honesty rules

MetroInsights is a planning prototype. Its demand targets and holdout are synthetic. Model metrics describe agreement with generated scenario labels, not real passenger counts or operational DMRC accuracy. Network topology, coordinates, capacities, headways, service frequency, and interchange splits are project data or configurable assumptions unless separately verified. Say “estimated” and explain the assumptions. Do not describe this as a live passenger-count, train occupancy, or production DMRC system.

Verified snapshot used in this edition

The cleaned station table contains 243 station records. The topology graph's destination count depends on the selected source and excludes the source itself; 242 destinations can be returned when all other modeled stations are connected. Do not say 226 or 228 without checking the current data. The project defines 11 route keys, including branch keys. The synthetic training series uses 243 stations x 728 dates x 19 hourly slots (05:00-23:00), or 3,361,176 rows. A chronological holdout begins 2026-10-01 and has 438,615 rows. The recorded HistGradientBoostingRegressor scores are MAE 26.58, RMSE 51.10, R2 0.9946, and MAPE 5.18%; Ridge MAE is 53.67 and historical station/day/hour mean MAE is 60.25. These are synthetic-only evaluation results.

Current stack and constraints

FastAPI + Uvicorn, Starlette/Jinja2, server-rendered HTML with JavaScript/CSS, Pandas, NumPy, scikit-learn, joblib, CSV/JSON, and a custom adjacency graph with heapq Dijkstra. No relational database, live ridership feed, authentication, or distributed job queue is present. The generated multi-million-row CSV is a training input; runtime inference loads compact serialized model/metadata plus station and operations inputs, rather than reading every training row on each request. Runtime startup imports routes and eagerly loads artifacts/builds the graph, so missing files can prevent startup. The current FastAPI migration has passed static compilation and diff checks in prior work, but an actual runtime smoke test was not possible in the offline environment; verify locally before claiming deployment verification.